

Codeforces 449B

Jzzhu and Cities

Dijkstra com análise de arestas redundantes

Grupo F

Integrantes: João Vitor Silva • Antonio Davi • Pablo Dornelles

Orientador: Prof. Me Ricardo Carubbi

Linguagem: Python 3 • Plataforma: Codeforces

1. O problema

Enunciado

n cidades, sendo a cidade **1** a capital.
 m estradas bidirecionais com peso x .
 k trens, cada um direto da capital a uma cidade s com custo y .

Objetivo

Fechar o máximo possível de trens sem alterar a distância mínima da capital a nenhuma cidade.

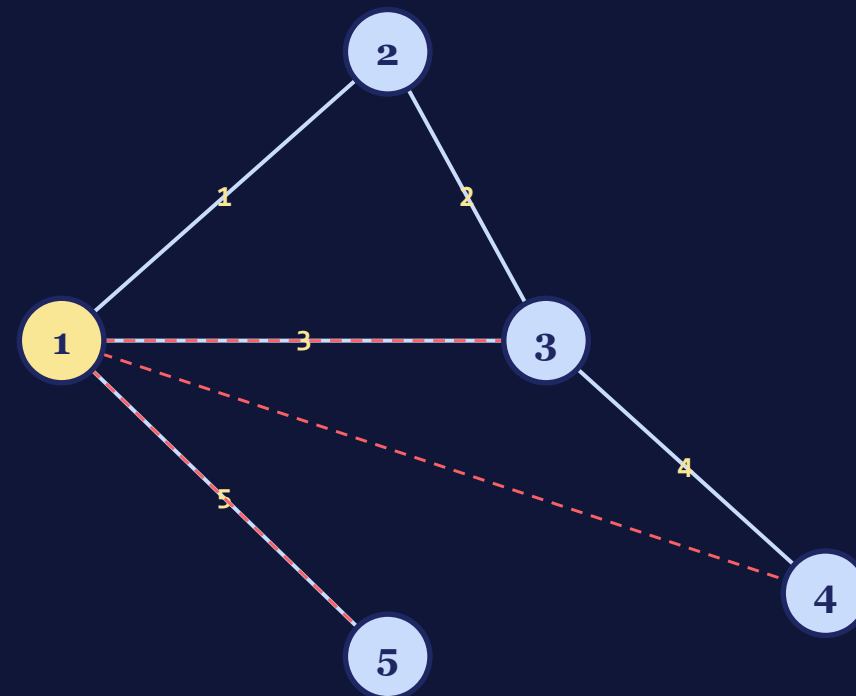
Saída para o exemplo:

2

trens podem ser fechados

Limites: $n \leq 10^5$ • $m \leq 3 \cdot 10^5$ • $k \leq 10^5$ • pesos até 10^9

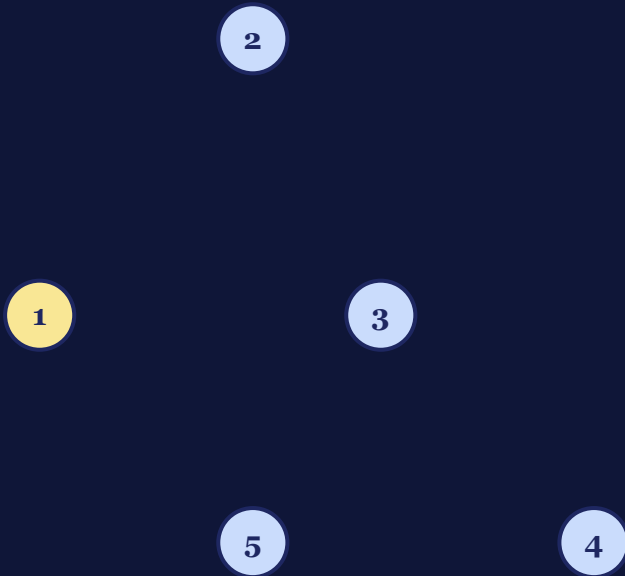
Exemplo 1



2. Do enunciado ao grafo

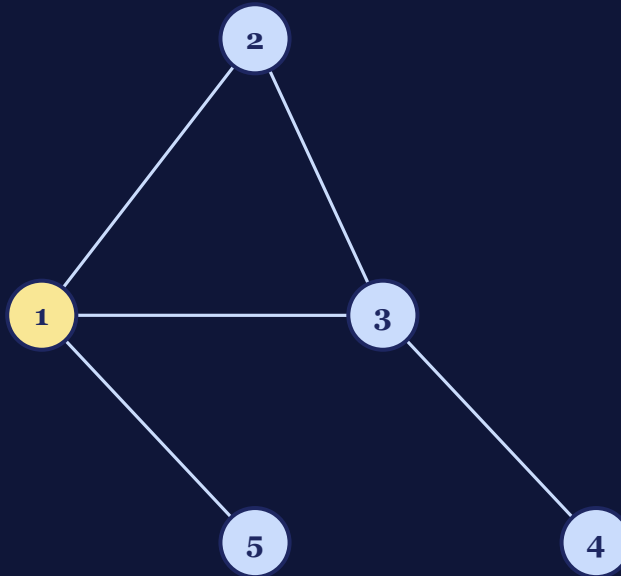
Cada linha da entrada vira um elemento do grafo. A imagem cresce em três etapas:

1. Cidades viram vértices



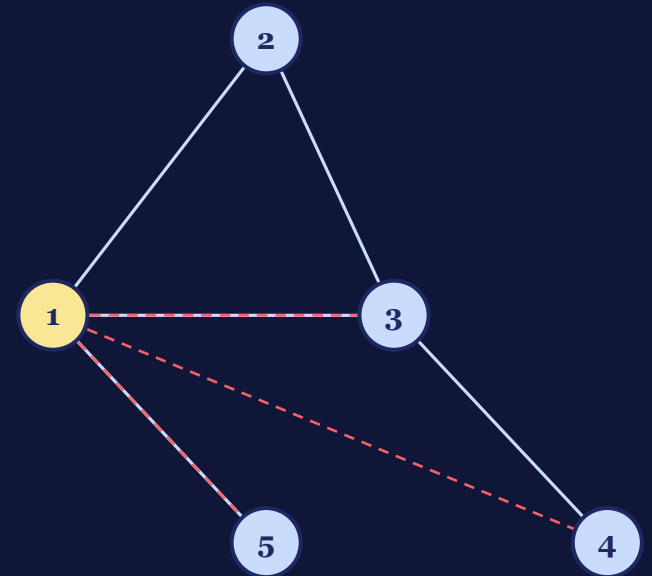
n nós no plano — cidade 1 é a capital (origem).

2. Estradas viram arestas



Cada estrada (u, v, x) é uma aresta bidirecional de peso x .

3. Trens viram arestas extras



Cada trem (s, y) é uma aresta da capital 1 para s , marcada como trem.

3. Modelagem como grafo

Correspondência enunciado ↔ grafo

Enunciado	Grafo $G(V, E)$
n cidades, capital = 1	n vértices, origem $v = 1$
Estrada (u, v, x) bidirecional	Aresta (u, v) com peso x — tipo: estrada
Trem (s, y) direto da capital	Aresta $(1, s)$ com peso y — tipo: trem
Distância mínima até cada cidade	Dijkstra de origem única a partir de 1

Por que Dijkstra é aplicável?

- **Pesos não negativos:** $x, y \geq 1$.
- **Caminho mais curto único a partir da capital:** origem única para todos os vértices.
- **Trens como arestas:** naturalmente integrados ao grafo, sem mudar o algoritmo.

$$V = n, E = 2m + k$$

Representação: lista de adjacência

`adj[u] = [ (v, peso, é_trem), ... ]`

- estradas: `adj[u].append((v, x, False))` e `adj[v].append((u, x, False))`
- trens: `adj[1].append((s, y, True))` (só saindo da capital)

4. Estratégia: Dijkstra + marcação de alternativa

Variação utilizada: Dijkstra clássico de origem única, com uma anotação por vértice.

Inicialização

```
dist[1] = 0
dist[v] = ∞  ∀v ≠ 1
via_estrada[1] = True
via_estrada[v] = False
pq = [(0, 1)]
```

Fila de prioridade

```
heapq (heap binário mín.)
extraí sempre o vértice
de menor distância.
Descarta entradas obsoletas
via  if d > dist[u]: continue
```

Critério de parada

```
fila vazia.
Ao final, dist[] tem o caminho
mínimo de 1 até cada vértice,
incluindo trens como arestas.
```

Relaxamento da aresta $u \rightarrow v$ (peso w , é_trem t)

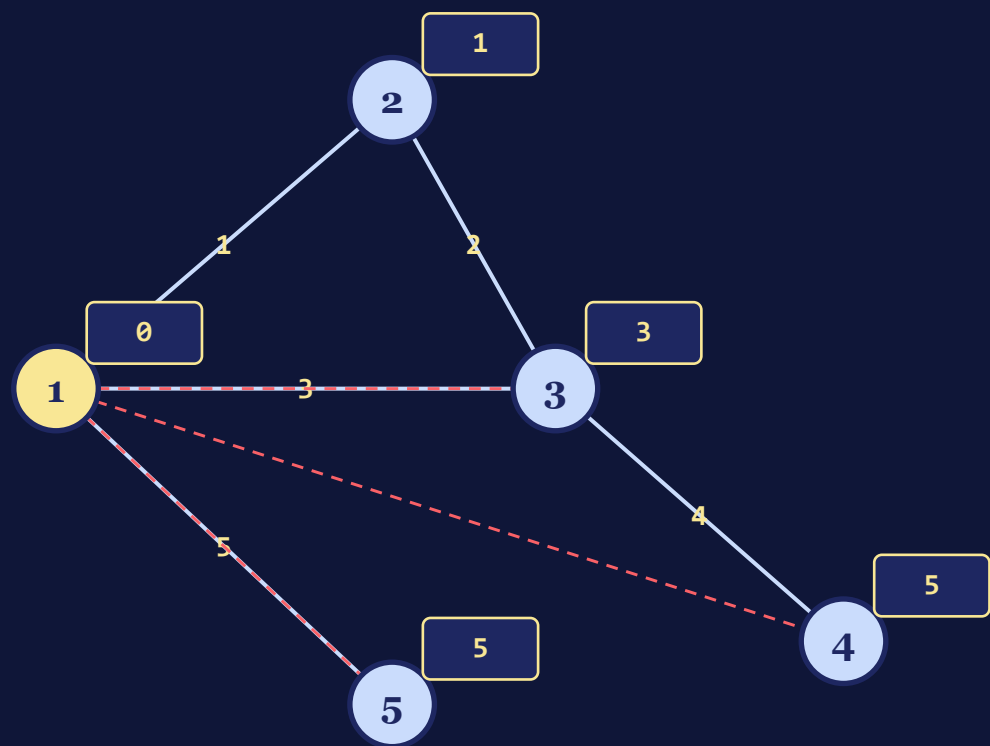
```
nd = dist[u] + w

se nd < dist[v]:                                # encontrou caminho mais curto
    dist[v] = nd; via_estrada[v] = (not t); push(pq, (nd, v))

senão se nd == dist[v] e not t:                  # alternativa de mesmo custo por estrada
    via_estrada[v] = True
```

5. Dijkstra em ação — Exemplo 1

Grafo + dist finais



Iterações (extração da fila)

pop	d	atualizações
1	0	dist[2]=1 dist[3]=3 dist[5]=5 dist[4]=5*
2	1	– (nd=3 == dist[3], via estrada → via_estrada[3]=T)
3	3	– (3→4 daria 7 > 5)
4	5	–
5	5	–

* dist[4]=5 via trem (1→4, y=5). via_estrada[4] = False.

Estado final dos vetores

dist = [_, 0, 1, 3, 5, 5]
via_estrada = [_, T, T, T, **F**, T]

→ apenas a cidade 4 depende de trem

6. Regras de fechamento dos trens

Observação-chave: todo trem sai da capital, então em qualquer caminho mínimo um trem aparece como a **primeira (e única) aresta**. Decidimos um trem por vez, agrupados pela cidade de destino.

Trem inútil

$$y > \text{dist}[s]$$

→ FECHA

O melhor caminho até s já é menor que o trem;
fechar não muda nada.

Alternativa por estrada

$$y = \text{dist}[s] \text{ e } \text{via_estrada}[s]$$

→ FECHA

Há um caminho ótimo cujo último passo é estrada
— manter o trem é redundante.

Trem essencial

$$y = \text{dist}[s] \text{ e } \neg \text{via_estrada}[s]$$

→ mantém 1, fecha duplicatas

Sem trem, $\text{dist}[s]$ aumentaria. Mantém-se
exatamente um trem com esse custo.

Aplicação no exemplo 1

```
trem (3, 5): y=5 > dist[3]=3           → fecha
trem (4, 5): y=5 = dist[4], ¬via_estrada[4] → mantém
trem (5, 5): y=5 = dist[5], via_estrada[5]  → fecha
```

7. Complexidade

Tempo

$$O((V + E) \cdot \log V)$$

$V = n$ (vértices)
 $E = 2m + k$ (estradas duplicadas + trens)
 $\log V$ (heap binário, heapq)

 $+ O(k)$ contagem final sobre os trens

*Cada vértice entra na fila no máximo uma vez por relaxamento bem-sucedido;
cada aresta é processada $O(1)$ vez na ramificação ativa.*

Memória

$$O(V + E)$$

- lista de adjacência $\approx 2m + k$ entradas
- vetores dist e via_estrada de tamanho n
- heap com no máximo $O(E)$ entradas vivas

Nos limites do problema

$n \leq 10^5$ $m \leq 3 \cdot 10^5$ $k \leq 10^5$
 $\Rightarrow \sim 7 \cdot 10^5$ arestas e $\sim 6 \cdot 10^6$ ops de heap.

Submissão real:

tempo: **1625 ms** (limite 2000 ms)
memória: **180 MB** (limite 256 MB)

8. Casos especiais

Pesos grandes (até 10^9)

Distâncias podem chegar a $\sim 10^{14}$. Em Python inteiros são arbitrários — sem overflow. Em Java, usar long.

Múltiplos trens para a mesma cidade

Agrupamos por destino. No máximo um é mantido quando estritamente necessário; os demais empatados são fechados.

Arestas paralelas entre o mesmo par

Tratadas naturalmente — cada uma vira uma entrada própria na lista de adjacência. O Dijkstra escolhe a melhor.

Empate trem × estrada ($y = \text{dist}[s]$)

A marcação `via_estrada[s]` detecta a alternativa por estrada e permite fechar o trem redundante.

Trem que melhora a distância ($y < \text{dist}[s]$)

Tratado dentro do próprio Dijkstra — o trem é apenas mais uma aresta. `dist[s]` cai e `via_estrada[s]` fica False.

Conectividade garantida

O enunciado garante que toda cidade alcança a capital, então nenhum `dist[v]` fica em ∞ no final.

Resultado

Accepted

#376408159

ID da submissão

1625 ms

tempo (limite 2 s)

180 MB

memória (limite 256 MB)

Em resumo

- Modelagem unificada: trens são apenas arestas extras a partir da capital.
- Uma única execução de Dijkstra com a marcação `via_estrada` já resolve.
- Contagem direta sobre os trens, agrupados por destino, fornece a resposta.